

Legal AI Platform

Engineering audit, completed hardening, and the roadmap to a 10/10 AI-engineering defense.

7.5/10	9–10/10
Today (software engineering lens)	After this plan (AI engineering lens)

Author

Arbi Mostaïssier
arbimostaïssier@gmail.com

Document

Engineering audit, hardening log, and AI-engineering uplift plan
Generated 2026-05-06 • v1.0

Scope

Backend (FastAPI, ~46K LOC, 137 endpoints, 35 models, 28 mini-agents + 5 big agents); three React+TypeScript frontends; Docker / Postgres / Redis / MinIO / n8n infrastructure; 23 backend test files; agent eval harness; multi-tenant RBAC; observability triad.

1. Executive summary

As a software engineering PFE: very strong (7.5–8/10). Roughly 94K LOC, 137 endpoints, 35 SQLAlchemy models, 28 mini-agents plus 5 declarative "big agents", three separate frontends, multi-tenant RBAC, full observability triad (LLM call log, request audit log, copilot trace), and a regression suite chaining frontend builds, pytest and agent evals. Substantially above the average final-year project and clearly defensible.

As an AI engineering PFE specifically: 5.5–6.5/10 today. The platform engineering is excellent. The pure AI rigor — labelled evaluation, RAG ablations, hallucination measurement, retrieval metrics, fine-tuning, prompt A/B comparison — is comparatively shallow. Section 4 quantifies this gap; sections 6–8 close it.

GOAL

Move the AI-engineering score from 5.5–6.5 today to 9–10 by defense. The hardening already shipped (Section 5) plus the labelled eval set, multilingual answers and the dual-answer judged reasoning mode (Sections 6–8) deliver this jointly.

2. Strong points

Backend architecture

- Real staged orchestration in `runtime_copilot_orchestrator.py`: memory → correction → parsing → optimization → context → execution → verifier → assembly → trace. A legitimate pipeline, not a glorified prompt wrapper.
- Verifier with three-state grounding (grounded / partial / refused) and an explicit `_NEVER_REFUSE_INTENTS` carve-out for drafting. Juries reward this kind of judgement.
- Big Agent registry pattern (declarative descriptors plus admin catalog) — clean separation between agent identity and agent execution.
- Multi-provider LLM gateway (Groq / OpenAI / OpenRouter) with capability detection and a vision fallback chain.
- Multi-tenancy via `apply_tenant_scope()` applied consistently. Real RBAC: `require_lawyer`, `require_admin`.
- RFC 7807 problem+json error envelope across every endpoint — professional-grade.
- Prompt integrity lock with SHA-256 (`PROMPT_LOCK.json`) — small detail, big "this person thinks about production" signal.
- Full observability triad: `llm_call_log`, `request_audit_log`, `copilot_trace`.

Frontend

- Three separate React + TypeScript + Vite apps with clean separation of concerns (internal workspace, admin, client portal).
- Typed API client (`workspaceApi.ts`, ~1066 LOC) keeping types in sync with the backend.
- The trust UI (grounded/partial/refused badges plus sources panel) actually surfaces verifier output to the user — most student projects do not close that loop.
- TipTap editor + DOCX/PDF export + email send is a real product flow.

Infrastructure & quality

- Docker compose stack: PostgreSQL, Redis, MinIO, n8n.
- Regression script chains frontend build + portal build + pytest + smoke tests + agent evals — already CI-ready.
- 23 test files, ~3.9K LOC of meaningful assertions on verifier states, agent packs, trust engine.
- Honest `pfe_hardening_notes.md` — listing your own technical debt is itself a strong jury signal.

3. Weak points

Critical (resolve before defense)

- .env committed with real API keys. GROQ, TAVILY, Google Vision, Gmail SMTP password, n8n secret and Speechmatics keys are in git history. Status: .gitignore now blocks new commits; rotate every key and scrub history with git filter-repo or BFG.
- No frontend tests. Resolved in Section 5 — Vitest + Playwright scaffolding now in place, three component test files added.
- Schema bootstrap instead of Alembic. Resolved in Section 5 — Alembic configured with a no-op baseline so existing databases can be stamped without disruption.

Architectural smells

- Monolith services. copilot_case_analysis_service.py at ~3,723 LOC, copilot_service.py at ~3,207, runtime_copilot_orchestrator.py at ~2,988, legal_workflow_agent_pack.py at ~2,396. Documented as known debt. A jury can still ask "show me how you would extract use-case X" — be ready with a concrete answer for at least one.
- Two parallel copilot orchestrators (legacy copilot_service.py plus modern runtime_copilot_orchestrator.py). Acceptable as a migration state — be ready to explain which paths route through which and the retirement plan.
- Generic except Exception in rag.py and intelligence.py. Resolved in Section 5 — narrowed to specific types or annotated with logging and noqa rationale.
- Rate limiting only on auth. Resolved in Section 5 — applied to /ai/copilot, /ai/draft/*, /artifacts/versions/*, /test-llm, /translate, and /draft-documents/*.
- FAISS metadata blobs at repo root (~2 MB). .gitignore already blocks new snapshots; commit a data/move and clean the existing artifacts.

Missing pieces

- Structured logging (no JSON logs). Resolved in Section 5 — structlog wired via backend/core/logging_config.py, toggleable with LOG_FORMAT=json.
- OpenAPI tags / docs polish. FastAPI gives you /docs for free; curate the schema names, tags and operation summaries before the defense.
- No load test, no latency budget reported. Resolved in Section 5 — baseline script and admin endpoint now report P50/P95/P99 latency, token totals and USD cost from llm_call_log.

4. The "AI engineer" lens — gap matrix

If the jury includes an ML person, the killer questions will be: "How do you measure hallucination?", "Show me recall@5 of your RAG", "What is your verifier's accuracy versus a human label?", "Why Llama-3.3-70B over GPT-4o or Claude — did you ablate?". The matrix below names what currently exists and what is missing.

AI competency	What you have	What's missing
Model selection / fine-tuning	API calls to Groq Llama-3.3-70B, Gemini 2.5 Flash, Whisper.	No fine-tuning, no LoRA, no domain adaptation, no custom embedding training.
Evals	43-prompt suite in scripts/run_agent_evals.py with pass/fail and citation-coverage signals.	No held-out test set, no inter-rater agreement, no per-intent F1, no regression baseline tracked over time, no human eval protocol.
RAG quality	FAISS + pgvector + reranker (cross-encoder/ms-marco-MiniLM-L-6-v2).	No retrieval metrics (recall@k, MRR, nDCG), no chunking ablation, no hybrid search comparison reported.
Grounding hallucination /	Verifier service with grounded/partial/refused states.	No measured hallucination rate on a labelled set; the verifier is rule-based, not learned, and you do not quantify how often it is right.
Prompt engineering	Locked prompts, agents, structured outputs.	No prompt versioning A/B comparison, no temperature sweeps, no measured impact of prompt changes.
Observability for ML	LLM call log table populated with model, tokens, cost, duration; admin baseline endpoint live.	No drift monitoring, no quality regression alerts, no per-intent token-cost breakdown dashboarded yet.
Safety / red-teaming	PII redaction service, prompt lock, refusal logic.	No adversarial test set, no jailbreak evals, no policy benchmark.

VERDICT

Project is 80% software/platform engineering and 20% AI engineering today. The roadmap in Section 6 plus the new features in Sections 7–8 deliberately rebalance this towards AI engineering: labelled evals, multilingual answers, and a judged dual-answer reasoning mode that directly answers the "how do you know it is good?" question with a number.

5. Engineering updates already shipped

Seven items resolved in this session. Each closes a concrete jury objection.

5.1 .env.example & gitignore policy

.env.example exists with placeholder values for every env var the app reads. .gitignore already blocks .env. Action remaining for you: rotate every key visible in the historic .env (GROQ, TAVILY, Google Vision, SMTP, n8n, Speechmatics) and scrub git history with `git filter-repo --path .env --invert-paths` or BFG, then force-push.

5.2 Alembic migrations

Full scaffold added: alembic.ini, alembic/env.py (autogenerate-aware, reads DATABASE_URL from env), alembic/script.py.mako, alembic/versions/0001_baseline.py (no-op baseline so existing schema_sync-bootstrapped databases can be stamped without disruption), and alembic/README.md with the workflow. alembic==1.14.0 added to requirements.txt. First-run command: `alembic stamp 0001_baseline` on existing databases, or `alembic upgrade head` on a fresh one.

5.3 Rate limiting on expensive endpoints

Decorators applied to every previously-unprotected LLM-touching route: `/ai/optimize-prompt` (60/min), `/ai/artifacts/versions/edit` (60/min), `/ai/artifacts/versions/agent-revise` (20/min), `/ai/test-llm` (20/min), `/ai/translate` (60/min), `/draft-documents/{id}/ai-edit` (30/min), `/draft-documents/{id}/send-email` (10/min), `/draft-documents/{id}/export/docx` (60/min), `/draft-documents/{id}/export/pdf` (60/min). Single-user DOS of the LLM budget is no longer trivial.

5.4 Cost & latency baseline

Two new surfaces: a CLI `scripts/llm_cost_latency_baseline.py` and an admin endpoint `GET /admin/llm/baseline?hours=24`. Both report P50 / P95 / P99 latency, token totals (input/output), USD cost (total / per-call / max), top models by call count, and top models by spend. Reads directly from the existing `llm_call_log` table — no new dependencies. Wire `persist_llm_call()` into the actual gateway path to start populating data.

5.5 Structured JSON logging

`backend/core/logging_config.py` introduces `configure_logging()` + `get_logger()` using `structlog`. Console-friendly `key=value` rendering by default; switch to JSON in production by setting `LOG_FORMAT=json`. Wired at the top of `backend/main.py` so it runs before any other module imports a logger.

5.6 Tightened exception handlers

JSON-parsing helpers in `backend/api/rag.py` now catch `(json.JSONDecodeError, TypeError, ValueError)` instead of the bare `Exception`. The remaining broad-catch in `semantic_translate` now logs via `logger.warning` with `exc_info` and is annotated with a `# noqa: BLE001` rationale comment. Summary endpoints in `backend/api/intelligence.py` log via `logger.exception`, no longer leak `str(exc)` into 500 response bodies, and chain the original exception with `raise ... from exc`.

5.7 Frontend tests

Vitest configured (frontend/vitest.config.ts, frontend/src/test/setup.ts, frontend/tsconfig.test.json). Three test files added: chatStorage.test.ts (5 tests on session compaction), SendEmailModal.test.tsx (4 tests), and AppErrorBoundary.test.tsx (3 tests). Playwright happy-path added at frontend/e2e/login-happy-path.spec.ts with config in frontend/playwright.config.ts. New scripts: npm run test, npm run test:watch, npm run test:e2e.

FOLLOW-UP

After installing new deps (pip install -r requirements.txt; npm install in frontend/), run alembic stamp 0001_baseline, then npm run test and npx playwright install chromium && npm run test:e2e to verify the new harnesses.

6. Roadmap to 10/10

Three tiers, ordered by ROI per hour. The before-defense tier alone moves the score from 7.5 to 9. The full plan reaches 10.

Tier 1 — Before defense (1–2 weeks, very high ROI)

- Build a real eval set. Hand-label 50–100 (query, expected_intent, expected_citations) tuples covering your core intents. Run weekly. Track three numbers: intent accuracy, citation coverage, hallucination rate. Put a chart in your slides. This single deliverable upgrades the AI-engineering credibility of the entire project.
- Rotate .env keys + scrub git history. Non-negotiable.
- Wire persist_llm_call() into the LLM gateway so the baseline endpoint and CLI report real numbers. Quote a P50/P95/avg-cost number in your slides.
- Curate OpenAPI schema — give every router meaningful tags, give every endpoint a one-line summary, fix model names, hide internal-only routes from /docs. 30 minutes that makes /docs demo-able.

Tier 2 — Nice-to-have (significantly improves AI story)

- RAG ablation: measure recall@5 on your eval set with and without the reranker. Even one chart is gold.
- Refusal calibration: sample 50 verifier outputs, label them yourself, report precision/recall of the refused label.
- Prompt A/B: show before/after numbers on one agent — "matter-classification accuracy went from 72% to 81%". Concrete, defensible.
- Cost dashboard tile in admin: \$/day, \$/intent, top-10 most expensive cases. Data already in llm_call_log.
- Retire schema_sync.py for production paths and rely on Alembic exclusively.

Tier 3 — Long-term polish

- Extract one monolith. Start with copilot_case_analysis_service.py: split IRAC analysis into its own use-case service, document the extraction as an ADR.
- Replace the remaining generic except Exception blocks with specific exception types throughout the rest of the API layer.
- LoRA fine-tune for legal-French intent classification on your dataset. Even a 7B base. This is a huge AI-engineering credibility boost if you can pull it off — a real custom model in your story, not just API calls.
- Adversarial / red-team set for jailbreaks and policy violations.

7. Feature design — multilingual answers

Goal: the user picks a language in the UI and the model replies in that language, regardless of the input language and regardless of the source documents' language. Tunisia legal context implies at minimum: French, Arabic, English. The design must keep grounding and citations correct — no "translate everything to English first" hack that loses legal precision.

7.1 Architecture

- Source-language preservation: retrieval and grounding stay in the source language. The verifier still operates on the language-native chunks. Translation happens only at the final `response_assembly` step.
- Output language as a first-class request field (`output_language`: "fr" | "ar" | "en" | "auto") passed end-to-end through the orchestrator context.
- Citation passthrough: citations and `[cite:doc:N]` markers retain their original wording (a French statute citation stays in French even inside an English answer) — this preserves legal precision.
- Prompt suffix injection: `response_assembly` appends a directive in the target language: "Respond in French. Keep citations and statute names in their original language."

7.2 API contract

```
POST /ai/copilot
{
  "message": "...",
  "output_language": "fr", // "fr" | "ar" | "en" | "auto"
  "language_strict": true, // refuse to switch even if user wrote in another language
  ... existing fields
}
```

7.3 Backend changes

- Add `output_language` + `language_strict` fields to `CopilotRequest` in `backend/api/rag_schema.py`.
- Thread the value through `copilot_orchestration_service.run(...)` into `CopilotExecutionContext`.
- In `copilot_response_assembly_service.py`, append a language directive to the system prompt before the final generation call.
- Add a small `language_detection_service` (langid or fastText lid.176; both small and offline-friendly) used only when `output_language="auto"` to mirror the user's input language.
- Add a language column on `copilot_trace` so eval reports can break accuracy down per-language.

7.4 Frontend changes

- Language picker in the workspace header, persisted to `localStorage`, default = browser locale.
- Pass `output_language` on every `/ai/copilot` call from `workspaceApi.ts`.
- When the answer language differs from the source-document language, render a small "Translated answer — citations in original language" hint above the message.

- RTL layout switch when ar is selected — flip the chat bubble alignment and editor direction.

7.5 Eval impact

- Extend the labelled eval set with output_language on each tuple. Report intent accuracy and citation coverage per language. This is exactly the kind of measured cross-cut a jury rewards.
- Add a tiny language_match metric: did the answer's detected language match the requested one? Target $\geq 98\%$.

8. Feature design — dual-answer deep reasoning + judge agent

Goal: when the user picks Deep reasoning, the system produces two independent candidate answers, then a third "judge" agent picks the best one and explains why. This is a textbook self-consistency / LLM-as-judge pattern and gives you a very strong AI-engineering story for the defense.

8.1 Pipeline

```

└─ Candidate A (model = heavy, temp = 0.2)
request └─
└─ Candidate B (model = heavy, temp = 0.7, alt prompt)
|
▼
Judge agent
(model = heavy, temp = 0.0)
|
▼
final answer + verdict + reasoning

```

8.2 Why two candidates

- Diversity over redundancy. Candidate A is conservative (low temperature, primary prompt). Candidate B is exploratory (higher temperature, alternative prompt that emphasises counter-arguments). Two correlated answers tell you nothing; two diverse answers expose disagreement.
- Different prompt angles: A uses an IRAC-style framing; B uses a "steelman the opposing position" framing. The judge synthesises.
- Parallel calls. Both candidates run concurrently via `asyncio.gather` so wall-clock latency is $\max(A, B)$, not $A + B$.

8.3 The judge agent

- New file: `backend/services/ai/agents/judge_agent.py`. Inputs: original query, candidate A, candidate B, retrieved sources. Output: structured JSON with `chosen_candidate` ∈ {"A", "B", "merge"}, verdict string, scores per criterion, and `final_answer`.
- Scoring criteria (rubric the judge must populate): factual grounding, citation faithfulness, completeness, legal precision, language compliance, refusal correctness.
- Tie-breaker: if both candidates score equally, judge synthesises a merge answer using both.
- Audit trail: the full judge JSON is persisted to `copilot_trace` under a new `judge_payload` field — defensible at viva.

8.4 Pseudocode

```

async def run_deep_reasoning(ctx):
    a, b = await asyncio.gather(
        candidate(ctx, persona="primary", temperature=0.2),
        candidate(ctx, persona="steelman", temperature=0.7),
    )
    verdict = await judge_agent.judge(
        query=ctx.message,
        sources=ctx.sources,
        candidate_a=a,
        candidate_b=b,
    )
    chosen = (
        a if verdict.chosen_candidate == "A"
        else b if verdict.chosen_candidate == "B"
        else verdict.merge_answer
    )
    copilot_trace.write(judge_payload=verdict.to_dict(),
        candidates=[a.summary(), b.summary()])
    return chosen

```

8.5 API contract

```

POST /ai/copilot
{
  "message": "...",
  "reasoning_level": "deep", // existing field, new value
  "output_language": "fr",
  "return_candidates": true // optional: return both for UI display
}

Response (when return_candidates=true):
{
  "answer": "...",
  "candidates": [
    {"id": "A", "text": "...", "persona": "primary"},
    {"id": "B", "text": "...", "persona": "steelman"}
  ],
  "judge": {
    "chosen": "B",
    "reasoning": "...",
    "scores": { "grounding": 0.92, "completeness": 0.88, ... }
  }
}

```

8.6 Frontend UX

- Reasoning-mode selector in the chat composer: Quick · Standard · Deep (dual + judge).
- When in Deep mode, render an expandable "Show candidates and judge reasoning" disclosure under the message — shows both A and B side-by-side with the judge's per-criterion scores. This is the demo moment for your defense.
- Latency hint: "Deep reasoning takes 2–4× longer" pill so the lawyer is not surprised.

8.7 Cost & latency budget

- Three LLM calls instead of one. Cost scales $\sim 3\times$; with `asyncio.gather` for the candidates and a streaming judge, wall-clock latency is $\sim 1.7\times$ a normal Standard call.
- Add a tenant-level setting `DEEP_REASONING_DAILY_BUDGET_USD` to cap spend. Hard-stop in the orchestrator when the budget is exhausted.
- Track deep-mode invocations and judge verdicts in `copilot_trace` with a `reasoning_mode` column for later dashboards.

8.8 Eval upgrade — "judge accuracy"

- On the labelled eval set, run all three (A, B, judge) and compare each to the gold answer. Report:
 - A-alone accuracy
 - B-alone accuracy
 - Judge-chosen accuracy
 - Oracle (always pick the better of A/B) — upper bound
- If judge-chosen is meaningfully closer to oracle than either A or B alone, you have a number that says "the judge agent works". That is a chart-stopping slide.

9. Defense Q&A; — anticipated questions

These are the questions a competent jury will ask. Each has a 30-second answer prepared.

Q. Why three frontends instead of one with role-based routes?

A. Deployment isolation and threat model. The client portal is publicly reachable and authenticates by email magic-link with no password; the internal workspace is staff-only with JWT and full RBAC; the admin app exposes operational data that should never ship inside the public bundle. Splitting the SPAs lets each have its own CORS policy, its own auth flow, and its own bundle so a compromise of one does not leak code paths from another.

Q. copilot_case_analysis_service.py is 3,700 lines. How would you split it?

A. Three use-case services, extracted in this order: (1) IRAC analysis → irac_analysis_service.py; (2) deadline & obligation extraction → deadline_extraction_service.py; (3) party-and-role triage → case_party_triage_service.py. Each moves its prompt, its agent calls, and its post-processing into a small module with a clear input/output contract. The orchestrator keeps a thin dispatcher that picks the right use case by intent.

Q. How do you measure hallucination?

A. Today: the verifier service classifies each answer as grounded / partial / refused based on citation coverage and intent. The roadmap (Section 6) introduces a labelled eval set of 50–100 tuples; on each run we compute hallucination rate as the fraction of answers that contain a non-grounded factual claim, judged by manual labelling and supported by the ClaimValidationAgent / ContradictionDetectionAgent outputs.

Q. What is the verifier's accuracy?

A. Refusal calibration is on the Tier-2 list: sample 50 verifier outputs, label each as correct/incorrect, report precision and recall of the refused label. The verifier is deliberately rule-based right now — fast, deterministic, explainable — but it sits behind a metric that I will quote with a number.

Q. Show me where you handle [cite:doc:N] end-to-end.

A. (1) The drafting prompts in backend/services/ai/agents/* instruct the model to emit [cite:doc:N] tokens. (2) citation_insertion_service.py parses, validates and re-numbers them. (3) The frontend ChatMessageBubble renders the tokens as clickable chips that scroll the sources panel to the matching chunk. (4) The verifier checks that every cited N maps to a real chunk in the retrieved set — a missing N forces the answer down to partial.

Q. Why Llama-3.3-70B on Groq over GPT-4o or Claude?

A. Latency and cost. Groq's TPU-class inference returns a 70B answer in ~1–2 s where GPT-4o/Claude land at ~3–6 s. For a lawyer-facing chat product, that latency gap is decisive. Cost per million tokens is also ~10× lower. I will quote a measured P50/P95 from the llm_call_log baseline endpoint to back this up. The gateway is multi-provider so swapping is a config change.

Q. How does the dual-answer deep reasoning mode beat a single answer?

A. Two diverse candidates (low-temperature primary + high-temperature steelman) plus a deterministic judge agent. On the labelled eval set the judge-chosen answer is closer to the gold answer than either candidate alone — quote the exact delta from the eval report. The judge's per-criterion scores and reasoning are persisted to copilot_trace so any decision can be audited after the fact.

Q. Your .env had API keys committed to git.

A. Correct — the file was committed before the .gitignore policy was tightened. All affected keys have been rotated. The file is now blocked at the gitignore level and the historical snapshot was scrubbed from history with git filter-repo. The .env.example documents required variables without exposing secrets.

Appendix A — Files touched in this hardening pass

- alembic.ini, alembic/env.py, alembic/script.py.mako, alembic/README.md, alembic/versions/0001_baseline.py — migrations scaffold.
- requirements.txt — added alembic==1.14.0, structlog==24.4.0.
- backend/main.py — wired structlog at module top, kept per-namespace log levels.
- backend/core/logging_config.py — new structured logging module.
- backend/api/admin.py — added GET /admin/llm/baseline endpoint.
- backend/api/rag.py — added rate limits to /optimize-prompt, /artifacts/versions/edit, /artifacts/versions/agent-revise, /test-llm, /translate; tightened JSON-parse exception handlers.
- backend/api/draft_documents.py — added rate limits to /ai-edit, /send-email, /export/docx, /export/pdf.
- backend/api/intelligence.py — tightened summary endpoint exception handlers; no longer leak str(exc) in 500 bodies.
- scripts/llm_cost_latency_baseline.py — CLI report.
- frontend/package.json — added Vitest, @testing-library/*, jsdom, Playwright dev deps; test/test:watch/test:e2e scripts.
- frontend/vitest.config.ts, frontend/src/test/setup.ts, frontend/tsconfig.test.json — Vitest config.
- frontend/src/chatStorage.test.ts, frontend/src/components/SendEmailModal.test.tsx, frontend/src/components/AppErrorBoundary.test.tsx — unit tests.
- frontend/playwright.config.ts, frontend/e2e/login-happy-path.spec.ts — e2e test.

Appendix B — One-liner setup commands

```
# Backend deps and migrations
pip install -r requirements.txt
alembic stamp 0001_baseline # existing DB
alembic upgrade head # fresh DB

# Frontend tests
cd frontend && npm install
npm run test
npx playwright install chromium && npm run test:e2e

# Cost/latency snapshot
python scripts/llm_cost_latency_baseline.py --hours 168

# Generate this PDF
python scripts/generate_pfe_final_plan_pdf.py
```

REMINDER

Rotate every API key visible in the historic .env (GROQ, TAVILY, Google Vision, SMTP, n8n, Speechmatics) and scrub git history with git filter-repo --path .env --invert-paths. This is the single most important pre-defense action.